

 Hierarchy & User Roles *seven roles + platform*

 **LEVEL 0 · PLATFORM**

 **Platform Owner — Super Admin**
Whhoohh Path LLP

- Creates and suspends tenant companies
- companyId is null — owns no employee record
- Enters a tenant only through support mode
- Sets each company's subscribed modules

 **LEVEL 1 · TENANT**

 **Company / Tenant**
e.g. Prozonic Technologies

- One company_id scopes every row in the system
- Own employees, payroll, letters and calendar
- Own SMTP credentials, encrypted before storage
- Own branding, leave types and designations

 **LEVEL 2 · ADMINISTRATION**

 **Company Administration**
Founder · Company Admin · HR Admin · Accounts Manager

- Founder / Company Admin — company profile, bank verification, every module
- HR Admin — people, payroll runs, letters, the seven company reports
- Accounts Manager — payroll and bank verification, not HR records
- bank.edit and bank.verify are held by different roles by design

 **LEVEL 3 · DELIVERY**

 **Project Manager · Team Lead**
Siblings, not a chain — each sits directly above Employee

- Project Manager — owns projects, re-scopes tasks, reads team reporting
- Team Lead — moves the board along, approves their reports' leave
- Neither reads salary, PAN, Aadhaar or bank details
- Scope follows reporting_manager, not seniority

 **LEVEL 4 · BASELINE**

 **Employee**
Every account has this baseline, whatever else it holds

- Biometric check-in and check-out, own attendance history
- Applies for leave, sees own balance and payslips
- Own letters, assets, tasks and the company calendar
- Reads no other employee's personal or financial data

 **How access is decided**
the two sentences this architecture rests on

- Two independent gates — the company must subscribe to the module AND the role must permit it.
- Every read is scoped by company_id before the role is consulted; a role widens access inside a tenant, never across one.
- Navigation guards hide; the database refuses.

 Deployed Stack *client → API → database*

 **CLIENT LAYER** *what the user runs*

 **Web Application**
React 19 · Vite · TanStack Query · shadcn/ui

- Portal modules, each role-guarded
- Strict CSP; connect-src pinned to the API
- Served as static files from Vercel

 **Mobile Application**
React Native · Expo · com.whhoohhpath.hrms

- Five tabs; twelve native modules
- ed25519 device key in the OS keystore
- Portal tab embeds the web app, signed in

 **Access** *Web browser · Android APK and iOS build via Expo EAS · one session, both surfaces*

HTTPS · REST + JSON · Bearer JWT on every request ▼

 **BACKEND LAYER** *Node.js 22 + Express 5 · hosted on Render*

 **Authentication & Authorization**
auth.ts · webauthn.ts · deviceauth.ts

- JWT issued at sign-in, 8 h TTL
- WebAuthn passkeys for the browser
- ed25519 challenge-response for the phone
- Five failures locks an address for 15 min

 **Session Context & Tenant Isolation**
db.ts — withSession()

- Every query runs inside a transaction
- user_id and company_id stamped on the connection first
- Connects as hrms_app, never the table owner
- No identity ⇒ the database returns nothing

 **Business Modules**
/query · /rest/:table · /rpc/:fn

- Reads answered from named views only
- Every write is a database function
- Payroll, leave, letters, tasks, reports
- The function re-checks the caller

 **Email Service**
Nodemailer · src/email

- Per-tenant SMTP, credentials encrypted at rest
- Templates, event triggers, delivery log
- Test endpoint; dev mode sends nothing

 **File Service**
POST /files · GET /files/:id

- Photos and logos held in Postgres (bytea)
- Read only after authentication, scoped by tenant
- 10 MB cap; no public bucket, no signed URLs

 **Rate Limiting & Audit**
security.ts · ISO 27001 A.8.5 / A.8.15 / A.8.16

- Separate limits for auth, API, admin and upload
- Security events written to append-only audit_logs

 **API routes + middleware** *requireAuth → rate limit → session context → validation → audit · /health · /auth · /query · /rest · /rpc · /files · /device · /webauthn · /email*
pg pool (max 10) · TLS verified · role hrms_app ▼

 **DATA LAYER** *Neon PostgreSQL — serverless · migrations 0001–0033*

 **Security in the database**
not in the app

- Row-level security on every tenant table
- Column grants withhold PAN, Aadhaar, bank
- Password policy enforced in SQL
- audit_logs immutable by trigger

 **Business logic**
SECURITY DEFINER functions

- apply_leave · decide_leave_request
- generate_payslip · generate_letter
- update_task — permission is per-field
- report_* — the seven company reports

 **Tables & storage**
one database, one backup

- companies · users · employees · attendance
- leave · salary structures · payslips · letters
- projects · tasks · assets · policies · messages · files (bytea)

 **INFRASTRUCTURE** *where each piece runs*

 **Vercel**
web SPA + headers

 **Render**
hrms-api, /health

 **Neon**
PostgreSQL + RLS

 **Expo EAS**
app builds

 **SMTP**
per tenant

 HRMS Modules *gated by subscription + role*

 **Mobile — 12 native modules** *plus Home, Check in and Messages*

 **People**
directory

 **Tasks**
board

 **Leaves**
apply · decide

 **Payslips**
own · all

 **Projects**
delivery

 **Attendance**
biometric

 **Reporting**
team view

 **Assets**
issued kit

 **Policies**
handbook

 **Letters**
five types

 **Calendar**
holidays

 **Announcements**
company

 **Portal — 16 modules in the WebView** *payroll runs · recruitment · reports · company bank · settings*

 Security & Compliance

 **Enforced in the platform**

- ✓ Tenant isolation by company_id
- ✓ Row-level security
- ✓ Role-based access control
- ✓ Column grants on sensitive fields
- ✓ Append-only audit logs
- ✓ SMTP credentials encrypted
- ✓ Rate limiting and login lockout
- ✓ CSP, HSTS, X-Frame-Options: DENY
- ✓ TLS end to end; no public file bucket
- ✓ JWT with a bounded lifetime

 EMAIL FLOW (SMTP) *outbound only*

 **HRMS API**
Nodemailer

 **Tenant SMTP**
Gmail · SendGrid · SES

 **Employee inbox**
payslip · letter · reset

⇒ The path of one request

A leave approval, followed end to end. Every other write in the system takes the same route.

1. The phone sends POST /rpc/decide_leave_request with the bearer token issued at sign-in.
2. requireAuth verifies the JWT signature and expiry, and puts userId and companyId on the request.
3. The API rate limiter checks the caller has not exceeded its window. A refusal is a 429 with Retry-After.
4. withSession opens a transaction and stamps the identity onto that connection before any statement runs. This is the step that makes row-level security work on Neon — the database has no other way to know who is asking.
5. The function name is checked against the allow-list. An unknown name never reaches Postgres.
6. decide_leave_request runs as SECURITY DEFINER. It re-checks that the caller may decide this request, adjusts the balance and writes the decision atomically.
7. The transaction commits, the identity is cleared from the connection, and it returns to the pool. A connection never carries the previous caller's identity.
8. The email service picks up the decision event and sends through the tenant's own SMTP account. Delivery is logged; a failure to send never rolls back the decision.

🔗 Why the roles are not a single ladder

Drawn as a chain, the hierarchy reads as though a Team Lead is a junior Project Manager and an Accounts Manager a junior HR Admin. Neither is true, and building permissions on that reading would hand each of them access they are deliberately denied.

Employee is a baseline that every account holds. Project Manager, Team Lead and Accounts Manager are three specialist branches off it, and the administration band is a fourth. Seniority does not decide scope — reporting_manager does. And the split between bank.edit (HR-level) and bank.verify (Founder, Company Admin, Accounts Manager) exists precisely so that no single person can both enter and approve a bank account.

☁ What is deployed where

Piece	Runs on	Notes
Web portal	Vercel	Static build of frontend/. Security headers and the /api rewrite come from vercel.json.
API	Render — hrms-api	Node 22, rootDir server/. Health check at /health. Secrets stored by Render, never in git.
Database	Neon PostgreSQL	Connected as hrms_app. The owner role is never used by the server — owners are exempt from RLS.
Mobile app	Expo EAS build	Android APK and iOS build. Points at the API's permanent address, not a developer machine.
Email	Tenant's SMTP provider	Gmail, Outlook, SendGrid or SES. Configured per company; credentials encrypted before storage.
Files	Inside Neon	Photos and logos as bytea. One backup covers the whole system.

⊖ Deliberately not in the stack

Two things commonly drawn on an architecture of this shape are absent, and their absence is a decision rather than an omission.

Not present	Why, and when it would change
Redis — queue, cache, rate limiting	Rate limiting runs in the API process, which is correct for one instance and wrong the moment a second one runs: the limits become per-instance. Email sends inline rather than through a queue. Redis is the first thing to add when the API scales past a single instance.
Object storage (S3 or equivalent)	Files are small images and there are as many as there are employees, so Postgres is simpler — one backup, one set of policies, no bucket whose permissions can drift. It stops being reasonable for bulk documents or attachments; move to object storage then and keep only the metadata in Neon.

❗ One correction to carry forward

The repository README still describes the earlier design — a React SPA talking straight to Supabase, with no application server. That is no longer what runs. The server in server/src replaced Supabase's PostgREST and GoTrue, and neon/migrations is the live schema; supabase/ is history. This sheet documents what is deployed. The README should be brought into line with it.